

Zelhoranka

Dwuosobowa gra karciano-planszowa — dokumentacja projektu

Aplikacja webowa 2D · widok prostopadle z góry · działa w pełni lokalnie · tryb kiosk
Wygenerowano automatycznie z danych projektu: 13.09.2026

1. Spis treści

1. Spis treści

2. Czym jest projekt

3. Uruchomienie

4. Zasady gry

5. Karty i reguły specjalne

6. Architektura
7. Odstępstwa od specyfikacji

8. Testy

9. Grafika i dźwięk

10. Tryb kiosk

11. Struktura repozytorium

2. Czym jest projekt

Zelhoranka to dwuosobowa gra karciano-planszowa osadzona w świecie fantasy (Zelhor, Aranor, Desilvar). Gracze werbują oddziały na szesnastopolową linię bitewną, zarządzają zaopatrzeniem, wydają rozkazy i rozstrzygają starcie. Wygrywa ten, kto wyprowadzi na tyły przeciwnika przełamanie o łącznej sile co najmniej 15.

Aplikacja jest grą webową 2D z widokiem prostopadle z góry, działa w pełni lokalnie i jest przystosowana do pracy w trybie kiosk na maszynie wirtualnej. Nie wymaga połączenia z siecią ani żadnej usługi zewnętrznej.

Realizacja wymagań

Wymóg	Realizacja
Mechanika planszowa	Tury i fazy, dobieranie kart, żetony oddziałów na planszy, zarządzanie zasobem (zaopatrzenie), rzut kostką przy losowaniu inicjatywy
Perspektywa top-down	Plansza 4×4 pola na gracza, rysowana płasko, kamera prostopadle z góry
Zakaz 3D i izometrii	Zero WebGL, zero <code>perspective</code> , zero <code>rotate3d</code> i <code>rotateX</code> . Dozwolone wyłącznie <code>translate</code> , <code>scale</code> , <code>rotate</code> w płaszczyźnie ekranu, <code>opacity</code> i filtry
Multimedia	Warstwa graficzna kart, żetonów i pól oraz warstwa dźwiękowa sterowana manifestem zdarzeń
Czystość kodu	<code>.gitignore</code> , skrypt pakujący, walidator danych, 73 testów jednostkowych

3. Uruchomienie

```
npm install
npm run dev          # tryb deweloperski, http://localhost:5173

npm run build        # sprawdzenie typów + build produkcyjny do dist/
npm run preview      # serwuje dist/ pod http://localhost:4173
```

Build używa ścieżek względnych, więc katalog `dist/` da się serwować z dowolnego podkatalogu lub otworzyć lokalnie.

Polecenie	Działanie
<code>npm test</code>	Testy jednostkowe silnika (Vitest)
<code>npm run validate</code>	Walidacja danych kart i lista brakujących grafik
<code>npm run placeholders</code>	Generuje tymczasowe grafiki zastępcze
<code>npm run docs</code>	Generuje ten dokument
<code>npm run pack</code>	Archiwum do oddania, bez <code>node_modules</code> i <code>dist</code>

4. Zasady gry

Plansza

Szesnaście pól, po osiem na gracza, w układzie 4 kolumny × 2 rzędy na stronę. Identyfikator pola to <gracz>-<rzęd><kolumna>, gdzie rząd F to linia przednia, a R tylna. Kolumny liczone są tak samo dla obu graczy, więc P1-F2 stoi naprzeciw P2-F2.

- **Flanka** — pola F0 i F3
- **Linia główna** — pola F1 i F2
- **Tylna linia** — pola R0 – R3

Na jednym polu stoi najwyżej jeden oddział. Kartę werbunku można zagrać wyłącznie na pole, którego strefa mieści się w dozwolonych strefach karty. Oddziały raz rozmieszczone nie przemieszczają się.

Przebieg tury

1. **Dobieranie.** Gracz uzupełnia rękę do 7 kart (twardy limit 8). W turze pierwszej wolno wymienić całą rękę, w kolejnych do 3 kart. Po wymianie dwóch można odrzucić jedną kartę i wymienić całą resztę, dopóki na ręce zostaje co najmniej jedna. Talia nie jest przetasowywana — jej wyczerpanie kończy partię.
2. **Logistyka.** Gracze zagrywają na przemian po jednej karcie. Zaopatrzenie działa jako pula: karty zaopatrzenia ją zasilają, z puli opłaca się koszty werbunku i zasadzek. Pula startuje od 1, a limit tury wynosi numer tury powiększony o 1. Pas kończy udział gracza w fazie; drugi może grać dalej sam.
3. **Strategia.** Gracze na przemian wydają rozkaz jednemu oddziałowi: atak, obrona albo postój. Rozkazy są jawne od chwili wydania i nie da się ich cofnąć. Przycisk „Reszta stoi” nadaje postój wszystkim niezadeklarowanym oddziałom naraz.
4. **Manewry.** Gracze na przemian zagrywają karty manewrów i odpalają zasadzki, aż obaj spasują z rzędu. Manewr modyfikuje deklaracje, nie planszę.
5. **Walka.** Starcie rozstrzyga się jednocześnie, na zamrożonej migawce stanu — żadna śmierć nie wpływa na obliczenia w tej samej fazie.

Zasięgi ataku

- FRONT — tylko przednie pole przeciwnika
- DEEP — przednie, a gdy puste, tylne
- RANGED — przednie albo tylne
- Oddział w tylnej linii atakuje tylko wtedy, gdy pozwala na to jego karta
- Strefy ograniczają rozmieszczanie, nie celowanie — oddział z flanki może uderzyć na ukos w linię główną

Rozstrzyganie starcia

1. Policz statystyki efektywne wszystkich żywych oddziałów.
2. **Wsparcie obrońców.** Każdy oddział z rozkazem obrony dokłada swoją defensywę jako obrażenia napastnikowi, który sam jest przez kogoś atakowany.

3. **Obrażenia.** Oddział ginie, gdy suma ofensyw napastników powiększona o wsparcie *przekracza* jego defensywę. Porównanie jest ostre — wartość równa nie wystarcza.
4. **Odwet obrońcy.** Obrońca dysponuje pulą równą swojej defensywie i zabija napastników w kolejności od największej ofensywy. Zabicie kosztuje ofensywę celu powiększoną o jeden. Pula niewystarczająca na największego napastnika przerywa odwet.
5. **Pojedynek wzajemny.** Gdy dwa oddziały atakują się nawzajem, wyższa ofensywa zabija niższą. Przy remisie obaj przeżywają. Zasada uzupełnia zwykłe liczenie obrażeń, nie zastępuje go.
6. Wszystkie oznaczone oddziały znikają naraz, a ich karty trafiają na stos odrzuconych.

Warunki zwycięstwa

Sprawdzone na koniec tury, po usunięciu poległych.

- **Przełamanie.** Dla każdej kolumny, w której przednie pole przeciwnika jest puste, sumujesz ofensywę własnych oddziałów z obu rzędów tej kolumny. Suma co najmniej 15 oznacza zwycięstwo. Obustronne przełamanie rozstrzyga wyższa suma, równa — remis.
- **Wyczerpanie talii.** Gdy talia któregoś z graczy się skończy, po zakończeniu bieżącej tury wygrywa gracz o większej łącznej ofensywie żywych oddziałów. Równość oznacza remis.
- **Limit 40 tur.** Bezpiecznik — rozstrzygnięcie porównawcze jak przy wyczerpaniu talii. Uzasadnienie w rozdziale 7.

5. Karty i reguły specjalne

Talia liczy **40 kart na gracza**, zbudowana z 16 definicji. Obaj gracze grają taliami identycznymi.

Typ karty	Definicji	Kart w talii
Werbunek	6	16
Zaopatrzenie	3	12
Zasadzka	2	4
Interwencja	2	4
Manewr	3	4
Razem	16	40

Oddziały

Nazwa	OFN	DEF	Koszt	Strefy	Zasięg	Reguła	Szt.
Piechota zelhorska	3	5	2	MAIN	FRONT	Oslona	4
Aranorscy jeźdźcy	5	2	3	FLANK	DEEP	Szarża	3
Łucznicy desilvarscy	3	2	2	REAR	RANGED · z tyłu	Ostrzał	3
Gwardia Zelhoru	4	6	4	MAIN, FLANK	FRONT	Niezlomność	2
Zwiadowcy Desilvaru	2	2	1	FLANK, REAR	FRONT	Zwiadowca	2
Kwatermistrz taboru	1	3	1	REAR	FRONT	Zaopatrzeniowiec	2

Reguły specjalne

Reguła	Działanie
Oslona	Sąsiedzi w tym samym rzędzie mają defensywę +1.
Szarża	W turze rozmieszczenia ofensywa +2.
Ostrzał	Atakuje z tylnej linii i sięga pól tylnych przeciwnika.
Niezlomność	Pierwsze oznaczenie śmierci w turze zostaje zignorowane.
Zwiadowca	Raz w turze możesz zmienić rozkaz tego oddziału w fazie manewrów.
Zaopatrzeniowiec	Karty zaopatrzenia zagrane przez właściciela wracają na rękę zamiast na stos odrzuconych.

Reguła to wpis w `rules.json` plus funkcja w `src/silnik/rules/`. Rejestr mapuje identyfikator na zestaw hooków, a brak implementacji dla zadeklarowanej reguły powoduje błąd przy starcie aplikacji — literówka ma boleć od razu, a nie zniknąć jako cichy brak efektu.

6. Architektura

Projekt trzymają trzy zasady, konsekwentnie egzekwowane w całym kodzie.

Silnik jest czysty

Katalog `src/silnik/` to wyłącznie TypeScript: zero DOM, zero importów z Reacta, zero `Math.random()`. Całość logiki to funkcja `reduce(stan, akcja) → { stan, zdarzenia }`. Losowość pochodzi z jednego generatora z ziarnem zapisanym w stanie gry, dzięki czemu rozgrywki są powtarzalne, testy stabilne, a przeciwnik komputerowy dostaje symulację starcia za darmo — wywołuje po prostu ten sam silnik na kopii stanu.

Interfejs nie zna zasad

Warstwa widoku renderuje stan i wysyła akcje. Nie liczy obrażeń, nie sprawdza legalności ruchu — pyta o to silnik funkcją `czyAkcjaLegalna()`. Stan interfejsu, taki jak zaznaczona karta czy podgląd zasięgu, trzymany jest osobno od stanu gry, który dzięki temu serializuje się do JSON bez śmieci.

Dane, nie kod

Karty, reguły specjalne, manifest dźwięków i parametry zasad siedzą w plikach JSON w katalogu `src/data/`. Dodanie karty to wpis w pliku, nie zmiana w komponencie. Silnik nie zawiera liczb magicznych — progi i limity czyta z `config.json`.

Zdarzenia napędzają multimedia

Reducer zwraca listę zdarzeń (`KARTA_ZAGRANA`, `JEDNOSTKA_ZGINELA`, `FAZA_ZMIENIONA` i inne). Warstwa mediów tylko je konsumuje: pojedynczy moduł zdejmuje zdarzenia z kolejki, odtwarza animację i dźwięk, a na czas odtwarzania blokuje wejście. Dodanie nowego dźwięku to wpis w manifeście, nie zmiana w logice.

7. Odstępstwa od specyfikacji

Specyfikacja pozostawiała dwanaście kwestii do rozstrzygnięcia. Wszystkie parametry wynikające z tych decyzji siedzą w `config.json`.

Kwestia	Rozstrzygnięcie
Talie	40 kart na gracza, obie identyczne
Zaopatrzenie	Model puli: karty zasilają pulę, z puli opłaca się koszty
Limit zaopatrzenia	Numer tury powiększony o bazę — inaczej karty zaopatrzenia byłyby bezużyteczne w turze pierwszej
Wolna droga na tyły	Kolumna z pustym przednim polem przeciwnika; liczą się oddziały z obu rzędów
Pojedynek wzajemny	Uzupełnia zwykle liczenie obrażeń, nie zastępuje go
Odwet obrońcy	Priorytet dla największych czytany dosłownie — brak środków na największego przerywa odwet
Ruch oddziałów	Nie istnieje; raz rozmieszczone stoją do śmierci
Remis	Istnieje przy równym przełamaniu i przy równej ofensywie

Pełna informacja zamiast tajności

Specyfikacja zakładała rozkazy tajne, deklarowane jednocześnie i rozdzielone ekranem zasłony. Projekt świadomie od tego odszedł: **ręce obu graczy są jawne, zasadzki widoczne, a zasłona usunięta**. Zakryta zostaje wyłącznie zawartość talii — symetrycznie, bo nie zna jej żadna ze stron.

Dlaczego wymagało to przebudowy fazy strategii. Sama jawność rąk jest symetryczna i nieszkodliwa. Problem leżał w deklaracji: przy zachowaniu modelu jednoczesnego, ale bez zasłony, gracz deklarujący jako drugi widziałby komplet rozkazów przeciwnika przed podjęciem własnych decyzji. To nie byłaby drobna przewaga, tylko rozstrzygnięcie partii.

Mechanizm	Cel
Deklaracja naprzemienna, po jednym oddziale	Przewaga informacyjna rozkłada się na całą fazę zamiast trafiać w całości do jednej strony
Rozkazu nie da się cofnąć	Inaczej obaj gracze czekali by na ruch przeciwnika i naprzemiennność traci sens
Pierwszy deklaruje gracz z większą liczbą oddziałów	Przy jawnej deklaracji pierwszeństwo jest obciążeniem; przewaga na planszy kosztuje przewagę informacyjną — mechanizm wyrównujący
Reguła zwiadowcy przebudowana	Podglądanie rozkazu stało się bezużyteczne; zwiadowca raz w turze koryguje własny rozkaz w fazie manewrów

Weryfikacja balansu. Dwieście partii rozegranych między przeciwnikami komputerowymi o równej sile, 189 rozstrzygniętych: gracz deklarujący jako drugi wygrał 100 razy (52,9%), deklarujący pierwszy 89 razy (47,1%). Odchylenie od równowagi wynosi 2,9 punktu procentowego przy błędzie losowym rzędu $\pm 3,6$ punktu, mieści się więc w granicy szumu. Średnia długość partii: 15,6 tury.

Limit tur jako bezpiecznik

Zasady ze specyfikacji nie gwarantują zakończenia partii. Gdy obaj gracze przestaną cokolwiek zagrywać, ręka nie schodzi, talia się nie wyczerpuje, a przełamanie nie rośnie — rozgrywka biegnie w nieskończoność. W trybie kiosk jest to niedopuszczalne, dlatego po 40 turach następuje rozstrzygnięcie porównawcze. W praktyce uruchamia się rzadko: w serii partii testowych zadziałał w około jednej szóstej przypadków, reszta kończyła się przełamaniem albo wyczerpaniem talii między czwartą a czternastą turą.

8. Testy

Testy jednostkowe (Vitest) obejmują silnik oraz test dymny warstwy widoku. Warstwa widoku renderowana jest przez `renderToString` z `react-dom`, bez dokładania biblioteki testującej ani środowiska przeglądarkowego do zależności projektu.

Obszar	Co sprawdza
Rozmieszczanie	Zgodność stref, zajętość pola, brak zaopatrzenia, wymiana kart przy duplikatach w ręce
Zasięgi	Zachowanie każdego z trzech trybów zasięgu, blokada ataku z tylnej linii, odrzucanie kolumn spoza planszy
Walka	Ostrość porównania, wsparcie obrońców, odwet z priorytetem, pojedynki wzajemny, reguły specjalne
Zwycięstwo	Próg przełamania dokładnie na granicy, wyczerpanie talii, limit tur
Determinizm	Powtarzalność przy tym samym ziarnie, serializacja stanu do JSON, spójność danych kart
Deklaracja	Naprzemienność, jednorazowość rozkazu, korekta zwiadowcy, jawność informacji
Przeciwnik komputerowy	Wyłącznie legalne akcje, dogrywanie partii do rozstrzygnięcia, powtarzalność
Warstwa widoku	Render każdego ekranu w każdej fazie bez wyjątku

9. Grafika i dźwięk

Żadna ścieżka do pliku nie występuje w kodzie komponentów — wszystko przechodzi przez `cards.json`, `rules.json` i manifest dźwięków. Nazwa pliku odpowiada identyfikatorowi karty.

Element	Rozmiar pliku	Rozmiar na ekranie
Awers karty	750 × 1050 px	150 × 210 px
Żeton oddziału	512 × 512 px	128 × 128 px
Pole planszy	512 × 512 px	128 × 128 px
Tło stołu	2560 × 1440 px	pełny ekran
Symbol reguły	SVG, kanwa 64 × 64	24 × 24 px

Statystyki nie są wypalone w grafice. Ofensywę, defensywę, koszt i symbole reguł rysuje aplikacja warstwą nad ilustracją. Liczby pozostają ostre w każdej skali, modyfikatory widać na żywo, a ilustrator nie musi numerować kilkudziesięciu plików.

Brak pliku nie psuje gry. Aplikacja szuka kolejno formatu docelowego, zastępnika i wreszcie rysuje placeholder proceduralny. Polecenie `npm run validate` wypisuje listę brakujących grafik, która jest zarazem listą zadań rysunkowych. Dźwięki działają tak samo: brak pliku oznacza ciszę, nie wyjątek.

10. Tryb kiosk

- Scena ma stałą proporcję 16:9 i skaluje się do rozmiaru okna, wyśrodkowana; w każdej rozdzielczości wygląda identycznie i nic się nie przewija
- Brak przewijania, zaznaczania tekstu, menu kontekstowego i przeciągania obrazków
- Cele kliknięcia nie mniejsze niż 44 px
- Zero natywnych dialogów przeglądarki — potwierdzenia to własne modale
- Automatyczny powrót do menu po 5 minutach bezczynności, żeby stacja demonstracyjna nie została zablokowana na porzuconej partii
- Muzyka startuje dopiero po pierwszej interakcji użytkownika, zgodnie z polityką autoodtwarzania w przeglądarkach
- Ustawienie `prefers-reduced-motion` zeruje czasy wszystkich animacji

11. Struktura repozytorium

```
src/
├─ data/          dane, nie kod: karty, reguły, dźwięki, konfiguracja zasad
├─ silnik/        czysty TypeScript – zero DOM, zero Reacta, zero Math.random()
│  └─ phases/     po jednym module na fazę tury
│  └─ rules/      implementacje reguł specjalnych
│  └─ __tests__/  testy jednostkowe
├─ ai/            przeciwnik komputerowy
├─ media/         audio, obrazki, wstępne wczytywanie zasobów
└─ ui/            warstwa widoku: ekrany, komponenty, style, teksty

tools/            walidator danych, generator grafik zastępczych,
                  generator dokumentacji, skrypt pakujący
public/assets/    grafiki, dźwięki, kroje pisma
```

Dokument wygenerowany poleceniem `npm run docs` na podstawie plików danych projektu. Źródłem prawdy dla zasad pozostaje `instrukcja.md`, dla parametrów – `src/data/config.json`.